Lecture => 4 Operators =>

Arithmetic -> +, -, /, *, %

Assignment -> =, +=, -=, /=, %=

Comparison -> ==, >=, <=

Conversion =>

```
let z = "120";
let a = Number(z);
console.log(typeof(a));
```

```
let a = "10";
let b = 10;
console.log(a==b);
```

-> JS convert string to number for comparison

-> true

-> in this type is ignored

Best

```
console.log(a===b);
```

-> before comparison of element it compare types of element should be done

Strictcheck ->

so a -> string type

b -> number type

as a === b -> return false

But what happened when]

```
let a = "10ab";
let b = Number(a);
console.log(b); → return NaN
```

→ type casting to number from string

↳ Not a number

because "10ab"

↳ having characters

Boolean → true for all values

→ false for 0 or "", undefined, null

Computer Science problem

```
let x = 0.1;
let y = 0.2;
let z = x + y;
console.log(z == 0.3); → false
```

But why

```
console.log(z); // → 0.30000000004
```

Why ↓

→ a fraction no. is stored in binary & convert into binary

0.25 == Binary Number

0.25 * 2 = 0.5 → 0

0.5 * 2 = 1.0 → 1

0.0 * 2 = 0 ___

→ 0.01

0.2 = = Binary number

$0.2 \times 2 = 0.4 \rightarrow 0$

$0.4 \times 2 = 0.8 \rightarrow 0$

$0.8 \times 2 = 1.6 \rightarrow 1$

$0.6 \times 2 = 1.2 \rightarrow 1$

$0.2 \times 2 = 0.4 \rightarrow 0$ ← same both

$0.00110011001100 \ldots$

∴ 0.2 is unable to store in exactly binary number

upto 10 decimal digit

decimal

$0.1 = 0.0001100110011 \rightarrow 0.100097656 25$

$0.2 = 0.0011001100110 \rightarrow 0.199218 75$

$0.0100110011 0 \qquad 0.299316 9062 5$